



Syntax Analysis



What Is Syntax Analysis?

After lexical analysis, your program becomes a sequence of tokens:

Code:

$x = a + b * 3;$

Token:

$ID(x) = ID(a) + ID(b) * NUM(3) ;$



What Is Syntax Analysis?

Now the syntax analyzer (also called parser) checks:

- Is the sequence of tokens arranged correctly?
- Does it follow the grammatical rules of the programming language?

It uses Context-Free Grammar (CFG) rules to verify this.



Why Do We Need Syntax Analysis?

A language (like C, Java, Python) has structure.

Example:

`"if condition then statement"` is valid

`"if then condition statement"` is invalid

The parser ensures the structure is correct. In short

- Lexical analyzer checks spelling.
- Syntax analyzer checks sentence structure.



What Is a Grammar?

A grammar defines valid sentences of a language.



Example of Grammar

$$E \rightarrow E + T \mid T$$

E stands for expression

$$T \rightarrow T * F \mid F$$

T stands for term

$$F \rightarrow (E) \mid id$$

F stands for factor

id is a variable (like x, y)

This grammar says expressions may contain:

- identifiers (id)
- parentheses
- and * operators
- nested expressions



How the Parser Uses Grammar

Suppose input tokens are:

`id + id * id`

It tries to verify the the tokens by creating a parse tree. If a it can create a valid parse tree from the grammar definition, the tokens are said to be in correct form. If the input cannot form a tree — the parser reports a syntax error.



Types of Parsers

There are two main types:

- Top-Down Parsers (start from start symbol)
- Bottom-Up Parsers (start from input and go to start symbol)



What is a Derivation?

A derivation is the step-by-step process of generating a string of terminals (a valid program statement) from the start symbol of a grammar by repeatedly applying productions.

- Starting from the grammar start symbol S
- Repeatedly replacing non-terminals using grammar rules
- Until only terminals remain in the string



Leftmost Derivation

In a leftmost derivation, at each step you expand the leftmost non-terminal first.

G:

$E \Rightarrow E + E \mid E * E \mid (E) \mid F$

$F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid \epsilon$

■ E

■ $\Rightarrow E * E$

■ $\Rightarrow F * E$

■ $\Rightarrow aF * E$

■ $\Rightarrow a * E$

■ $\Rightarrow a * (E)$

■ $\Rightarrow a * (E + E)$

■ $\Rightarrow a * (F + E)$

■ $\Rightarrow a * (aF + E)$

■ $\Rightarrow a * (abF + E)$

■ $\Rightarrow a * (ab + E)$

■ $\Rightarrow a * (ab + F)$

■ $\Rightarrow a * (ab + 1F)$

■ $\Rightarrow a * (ab + 10F)$

■ $\Rightarrow a * (ab + 10)$

■ E

■ $\Rightarrow E * E$

■ $\Rightarrow E * (E)$

■ $\Rightarrow E * (E + E)$

■ $\Rightarrow E * (E + F)$

■ $\Rightarrow E * (E + 1F)$

■ $\Rightarrow E * (E + 10F)$

■ $\Rightarrow E * (E + 10)$

■ $\Rightarrow E * (F + 10)$

■ $\Rightarrow E * (aF + 10)$

■ $\Rightarrow E * (abF + 0)$

■ $\Rightarrow E * (ab + 10)$

■ $\Rightarrow F * (ab + 10)$

■ $\Rightarrow aF * (ab + 10)$

■ $\Rightarrow a * (ab + 10)$



Why do we need Leftmost vs Rightmost derivations?

Top-down parsers use leftmost derivations

Bottom-up parsers use rightmost derivations in reverse



Ambiguity

If a string has:

- More than one leftmost derivation, or
- More than one rightmost derivation, or
- More than one parse tree

→ grammar is ambiguous.



Classical Ambiguous Grammar (Dangling-Else Problem)

Consider this grammar for conditional statements:

```
S → if E then S  
    | if E then S else S  
    | other
```

Let's parse

```
if E1 then if E2 then S1 else S2
```



Classical Ambiguous Grammar (Dangling-Else Problem)

else matches the inner if

```
if E1 then ( if E2 then S1 else S2 )
```

else matches the outer if

```
( if E1 then if E2 then S1 ) else S2
```

Both interpretations are valid according to the grammar — this makes the grammar ambiguous.



Why this happens

The grammar does not specify precedence of else — whether it binds to the closest preceding unmatched if, or to any outer one.

Languages like C, Java, Python, JavaScript follow this rule:

An else is matched with the nearest unmatched if.

A grammar must be rewritten to enforce this.



Left Recursion

A grammar is left recursive if:

$$A \rightarrow A \alpha$$

What is the problem here? Hint: Leftmost derivative.



Types of Left Recursion

- Immediate Left Recursion
 - A non-terminal refers to itself directly on the left side of the RHS.
 - Example: $A \rightarrow A\alpha \mid \beta$
- Indirect Left Recursion
 - A refers to itself through another non-terminal.
 - Example:

$$A \rightarrow B \alpha$$

$$B \rightarrow A \beta$$



Removing Immediate Left Recursion

$$A \rightarrow A\alpha \mid \beta$$

Remove left recursion by rewriting as

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$



Example

Remove left recursion

$$E \rightarrow E + T \mid T$$



Example of Indirect Left Recursion

$$A \rightarrow B a \mid c$$

$$B \rightarrow A b \mid d$$

Convert it into immediate left recursion and then remove